IPP - Practical File 2

Name: Swanand Wirkar
Course: BDA
Enrollment Number: 20251051

Question: Problem 1: Comparative Edge Detection on Natural Images. Download any outdoor natural image (trees, buildings, roads). Apply Sobel, Laplacian, and Canny edge detection. Compare the outputs and write a short analysis explaining which operator captures the most meaningful edges and why.

Code:

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("outdoor.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
canny = cv2.Canny(gray, 100, 200)
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].imshow(gray, cmap="gray")
axes[0, 0].set_title("Original Grayscale")
axes[0, 1].imshow(sobel, cmap="gray")
axes[0, 1].set_title("Sobel")
axes[1, 0].imshow(abs(laplacian), cmap="gray")
axes[1, 0].set_title("Laplacian")
axes[1, 1].imshow(canny, cmap="gray")
axes[1, 1].set_title("Canny")
for axis in axes.flat:
    axis.axis("off")
plt.tight_layout()
plt.show()
```

Question: Problem 2: Noise Sensitivity Experiment. Take a clean grayscale image and intentionally add Gaussian noise and salt-and-pepper noise. Apply Sobel and Laplacian on both noisy versions and evaluate which operator is more affected by noise, how edge clarity changes, and what preprocessing step improves the results.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

gray = cv2.imread("clean_image.jpg", cv2.IMREAD_GRAYSCALE)
gaussian_noise = np.random.normal(0, 20, gray.shape)
gaussian = np.clip(gray.astype(np.float32) + gaussian_noise, 0, 255).astype(np.uint8)
salt_pepper = gray.copy()
random_values = np.random.rand(*gray.shape)
salt_pepper[random_values < 0.02] = 0
salt_pepper[random_values > 0.98] = 255
images = [gray, gaussian, salt_pepper]
titles = ["Original", "Gaussian Noise", "Salt-and-Pepper Noise"]
fig, axes = plt.subplots(3, 3, figsize=(10, 9))
for row, current in enumerate(images):
    sobel_x = cv2.Sobel(current, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(current, cv2.CV_64F, 0, 1, ksize=3)
    sobel = cv2.magnitude(sobel_x, sobel_y)
    laplacian = cv2.Laplacian(current, cv2.CV_64F)
    axes[row, 0].imshow(current, cmap="gray")
    axes[row, 0].set_title(titles[row])
    axes[row, 1].imshow(sobel, cmap="gray")
    axes[row, 1].set_title("Sobel")
    axes[row, 2].imshow(np.abs(laplacian), cmap="gray")
    axes[row, 2].set_title("Laplacian")
for axis in axes.flat:
    axis.axis("off")
plt.tight_layout()
plt.show()
filtered_gaussian = cv2.GaussianBlur(gaussian, (5, 5), 0)
filtered_salt_pepper = cv2.medianBlur(salt_pepper, 5)
```

Question: Problem 3: Compression Ratio Comparison (RLE vs Huffman). Load a grayscale image with large smooth regions, such as a document scan or sky image. Compress it using Run-Length Encoding and Huffman Coding separately. Calculate bit-length before compression, bit-length after compression, and percentage reduction. Explain why each technique performed the way it did.

Code:

```
import cv2
import numpy as np
from collections import Counter
import heapq

image = cv2.imread("document.png", cv2.IMREAD_GRAYSCALE)
pixels = image.flatten().tolist()
original_bits = len(pixels) * 8
runs = []
for pixel in pixels:
    if runs and runs[-1][0] == pixel:
        runs[-1][1] += 1
    else:
        runs.append([pixel, 1])
rle_bits = sum(8 + 32 for pixel, count in runs)
frequencies = Counter(pixels)
heap = [[frequency, [symbol, ""]] for symbol, frequency in frequencies.items()]
heapq.heapify(heap)
if len(heap) == 1:
    codes = {heap[0][1][0][0]: "0"}
else:
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        for item in left[1]:
            item[1] = "0" + item[1]
        for item in right[1]:
            item[1] = "1" + item[1]
        heapq.heappush(heap, [left[0] + right[0], left[1] + right[1]])
    codes = {symbol: code for symbol, code in heap[0][1]}
huffman_bits = sum(frequencies[symbol] * len(codes[symbol]) for symbol in frequencies)
rle_reduction = (original_bits - rle_bits) / original_bits * 100
huffman_reduction = (original_bits - huffman_bits) / original_bits * 100
print("Original bits:", original_bits)
print("RLE bits:", rle_bits)
print("RLE reduction (%):", round(rle_reduction, 2))
print("Huffman bits:", huffman_bits)
print("Huffman reduction (%):", round(huffman_reduction, 2))
```

Question: Problem 4: JPEG Block Artifact Analysis. Implement an 8×8 block-based JPEG-style compression using DCT and quantisation. Compress the same image using low, medium, and high quantisation levels. Analyse where block artifacts appear, why textures lose detail, and how PSNR and SSIM change at each level.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
image = cv2.imread("texture.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)
height, width = image.shape
height = height - height % 8
width = width - width % 8
image = image[:height, :width]
base_quant = np.array([[16, 11, 10, 16, 24, 40, 51, 61],
                       [12, 12, 14, 19, 26, 58, 60, 55],
                       [14, 13, 16, 24, 40, 57, 69, 56],
                       [14, 17, 22, 29, 51, 87, 80, 62],
                       [18, 22, 37, 56, 68, 109, 103, 77],
                       [24, 35, 55, 64, 81, 104, 113, 92],
                       [49, 64, 78, 87, 103, 121, 120, 101],
                       [72, 92, 95, 98, 112, 100, 103, 99]], dtype=np.float32)
levels = {"Low": 0.5, "Medium": 1.0, "High": 2.0}
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
axes[0].imshow(image, cmap="gray", vmin=0, vmax=255)
axes[0].set_title("Original")
for axis in axes:
    axis.axis("off")
for index, (name, scale) in enumerate(levels.items(), start=1):
    quant_table = base_quant * scale
    compressed = np.zeros_like(image)
    for y in range(0, height, 8):
        for x in range(0, width, 8):
            block = image[y:y + 8, x:x + 8] - 128
            dct_block = cv2.dct(block)
            quantized = np.round(dct_block / quant_table)
            restored = cv2.idct(quantized * quant_table) + 128
            compressed[y:y + 8, x:x + 8] = restored
    compressed = np.clip(compressed, 0, 255).astype(np.uint8)
    reference = image.astype(np.uint8)
    psnr = peak_signal_noise_ratio(reference, compressed, data_range=255)
    ssim = structural_similarity(reference, compressed, data_range=255)
    axes[index].imshow(compressed, cmap="gray", vmin=0, vmax=255)
    axes[index].set_title(f"{name}\nPSNR={psnr:.2f}\nSSIM={ssim:.3f}")
    print(name, "PSNR:", round(psnr, 2), "SSIM:", round(ssim, 3))
plt.tight_layout()
plt.show()
```

Question: Problem 5: Texture-Based Segmentation using GLCM Features. Load a synthetic or natural image containing multiple textures, such as wood, fabric, leaves, or tiles. Extract GLCM features (contrast, energy, homogeneity) for each pixel neighbourhood. Segment the image into k = 3 clusters using these features. Write observations on how feature choice influences segmentation quality.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import graycomatrix, graycoprops
from sklearn.cluster import KMeans

image = cv2.imread("textures.jpg", cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (128, 128))
quantized = (image // 16).astype(np.uint8)
radius = 2
padded = cv2.copyMakeBorder(quantized, radius, radius, radius, radius,
cv2.BORDER_REFLECT)
features = []
for y in range(radius, image.shape[0] + radius):
    for x in range(radius, image.shape[1] + radius):
        patch = padded[y - radius:y + radius + 1, x - radius:x + radius + 1]
        glcm = graycomatrix(patch, distances=[1], angles=[0], levels=16, symmetric=True,
normed=True)
        contrast = graycoprops(glcm, "contrast")[0, 0]
        energy = graycoprops(glcm, "energy")[0, 0]
        homogeneity = graycoprops(glcm, "homogeneity")[0, 0]
        features.append([contrast, energy, homogeneity])
features = np.array(features, dtype=np.float32)
mean = features.mean(axis=0)
std = features.std(axis=0)
std[std == 0] = 1
scaled = (features - mean) / std
labels = KMeans(n_clusters=3, random_state=42, n_init=10).fit_predict(scaled)
segmented = labels.reshape(image.shape)
fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].imshow(image, cmap="gray")
axes[0].set_title("Original Image")
axes[1].imshow(segmented, cmap="viridis")
axes[1].set_title("GLCM Segmentation (k=3)")
for axis in axes:
    axis.axis("off")
plt.tight_layout()
plt.show()
```

Question: Problem 6: Multi-Feature Segmentation for Disease Detection. Load a plant or skin lesion image. Construct a feature vector combining intensity, gradient magnitude (Sobel), and local texture variance. Segment the image using K-means or Mean Shift. Evaluate which feature contributed most to separating diseased versus healthy regions, any misclassifications, and why they occurred.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

image = cv2.imread("plant_leaf.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
gray_float = gray.astype(np.float32)
sobel_x = cv2.Sobel(gray_float, cv2.CV_32F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray_float, cv2.CV_32F, 0, 1, ksize=3)
gradient = cv2.magnitude(sobel_x, sobel_y)
mean = cv2.blur(gray_float, (7, 7))
mean_square = cv2.blur(gray_float ** 2, (7, 7))
variance = np.maximum(mean_square - mean ** 2, 0)
features = np.stack([gray_float, gradient, variance], axis=-1).reshape(-1, 3)
minimum = features.min(axis=0)
maximum = features.max(axis=0)
scaled = (features - minimum) / (maximum - minimum + 1e-8)
labels = KMeans(n_clusters=3, random_state=42, n_init=10).fit_predict(scaled)
segmented = labels.reshape(gray.shape)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(image)
axes[0].set_title("Original Image")
axes[1].imshow(gray, cmap="gray")
axes[1].set_title("Grayscale")
axes[2].imshow(segmented, cmap="viridis")
axes[2].set_title("K-means Segmentation")
for axis in axes:
    axis.axis("off")
plt.tight_layout()
plt.show()
```